

Grafana and Prometheus



In the fast-evolving world of Generative AI, Large Language Models (LLMs) require robust monitoring to ensure performance, reliability, and cost-effectiveness. This is where **Grafana** and **Prometheus** shine as a powerful duo for observability in **LLMOps** (LLM Operations).

Why Monitoring Matters in LLMOps

Deploying and maintaining LLMs involves handling real-time inference, scaling, latency issues, and optimizing GPU usage. Without monitoring, teams risk unexpected downtimes, resource inefficiencies, and degraded model performance. **Grafana and Prometheus** provide a scalable solution to track these metrics and ensure a smooth AI pipeline.

Prometheus: The Time-Series Database for LLMOps

Prometheus is an open-source monitoring and alerting system optimized for time-series data. It collects real-time metrics and provides flexible querying for AI model performance.

Key Features:

- **Pull-Based Metrics Collection:** Prometheus scrapes metrics from LLM inference endpoints, API servers, and GPU usage stats.
- **Multi-Dimensional Data Model:** Enables efficient querying using PromQL (Prometheus Query Language).
- **Alerting and Rule-Based Monitoring:** Triggers alerts for latency spikes, GPU overheating, or memory overflows.
- **Long-Term Storage Integrations:** Works with remote storage solutions for extended logging.

Example

For an AI inference server running with FastAPI and an LLM model like Llama 3, we can expose an endpoint (`/metrics`) that Prometheus scrapes. Create a `main.py` file

```

from prometheus_fastapi_instrumentator import
Instrumentator
from fastapi import FastAPI

app = FastAPI()
Instrumentator().instrument(app).expose(app)

@app.get("/")
def read_root():
    return {"response": "Hello"}

@app.post("/predict")
def read_root():
    response = None # Call your LLM here
    return response

```

This enables Prometheus to collect API metrics automatically.

Setting Up Prometheus with Docker

To quickly deploy Prometheus and Grafana, use the following docker-compose.yml:

```

services:
  prometheus:
    image: prom/prometheus:latest
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
    ports:
      - "9090:9090"

```

Create a prometheus.yml file:

```
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'fastapi'
    static_configs:
      - targets: ['host.docker.internal:8000']
```

To create custom Prometheus metrics, visit this [link](#).

Grafana: The Visualization Layer for Observability

Grafana is an open-source analytics and monitoring platform that seamlessly integrates with Prometheus to provide real-time dashboards.

Why Grafana for LLMOps?

- **Rich Visualization:** Plot metrics like inference latency, GPU utilization, and memory consumption.
- **Custom Dashboards:** Create AI-specific dashboards to monitor token generation speed, API request counts, and user query distribution.
- **Alerting and Notifications:** Integrate with Slack, email, or PagerDuty for proactive issue resolution.
- **Data Correlation:** Compare multiple metrics to diagnose performance bottlenecks.

How to create a Grafana Dashboard from an Existing Prometheus Server

1. Open Grafana (<http://localhost:3000> with default admin/admin credentials).
2. Go to **Configuration > Data Sources** and select **Prometheus**.

3. Set the **URL** as `http://prometheus:9090` and save.
4. Create a new dashboard and add visualizations

Add Grafana to Docker compose

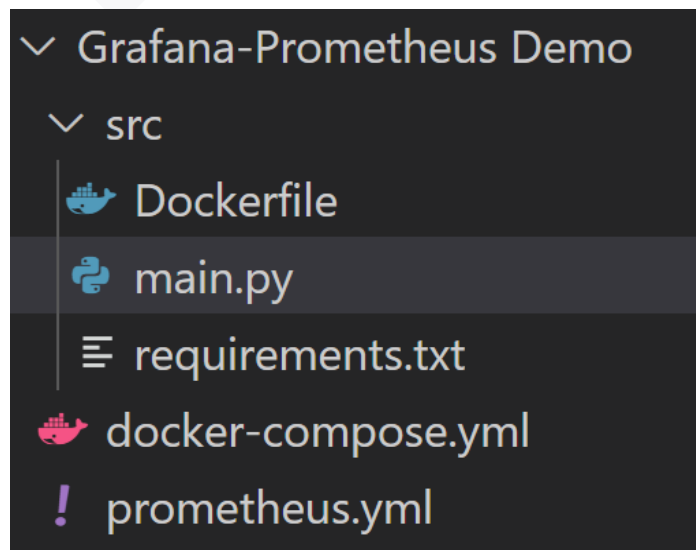
```
grafana:
  image: grafana/grafana:latest
  container_name: grafana
  ports:
    - "3000:3000"
  volumes:
    - grafana-data:/var/lib/grafana
  environment:
    - GF_SECURITY_ADMIN_PASSWORD=admin

volumes:
  grafana-data:
```

Persisting Grafana Data

By mounting a volume (`grafana-data`), Grafana dashboards and settings persist across container restarts, preventing data loss.

At this point, re-structure the directory structure into this format,



Download the other files from [here](#).

After restructuring and downloading necessary files, run this command

```
docker-compose up -d
```

Make sure Docker is installed on your system.

```
=> [fastapi internal] load metadata for docker.io/library/python:3.10-slim 1.9s
=> [fastapi 1/5] FROM docker.io/library/python:3.10-slim@sha256:f680fc3f447366d9be2ae53dc7a6447fe9b33311af209225783 0.0s
=> [fastapi internal] load build context 0.0s
=> => transferring context: 94B 0.0s
=> CACHED [fastapi 2/5] RUN ln -s /usr/bin/python3 /usr/bin/python 0.0s
=> CACHED [fastapi 3/5] WORKDIR /app 0.0s
=> CACHED [fastapi 4/5] COPY . . 0.0s
=> CACHED [fastapi 5/5] RUN pip install -r requirements.txt 0.0s
=> [fastapi] exporting to image 0.1s
=> => exporting layers 0.0s
=> => writing image sha256:3e6768b315387de62f855d0df2e0785effe751a538e7e4fdeb8d01b155cf31f3 0.1s
=> => naming to docker.io/library/grafana-prometheusdemo-fastapi 0.0s
[+] Running 3/3
✓ Container prometheus-demo Started 0.9s
✓ Container grafana-demo Started 0.9s
✓ Container fastapi-demo Started 0.8s
```

Open your Docker Desktop and go to Containers to view your running containers

☐	 grafana-prometheusdemo		Running (3/3)	1.69%	60 seconds ago				
☐	 grafana-demo a72ce5ff18cb 	grafana/grafana:latest	Running	1.42%	3000:3000 	60 seconds ago			
☐	 fastapi-demo f4d6ef6530f8 	grafana-prometheusdemo-fastapi	Running	0.27%	8000:8000 	60 seconds ago			
☐	 prometheus-demo 89436e72db2b 	prom/prometheus:latest	Running	0%	9090:9090 	60 seconds ago			

This shall create everything we need for this demo. Access

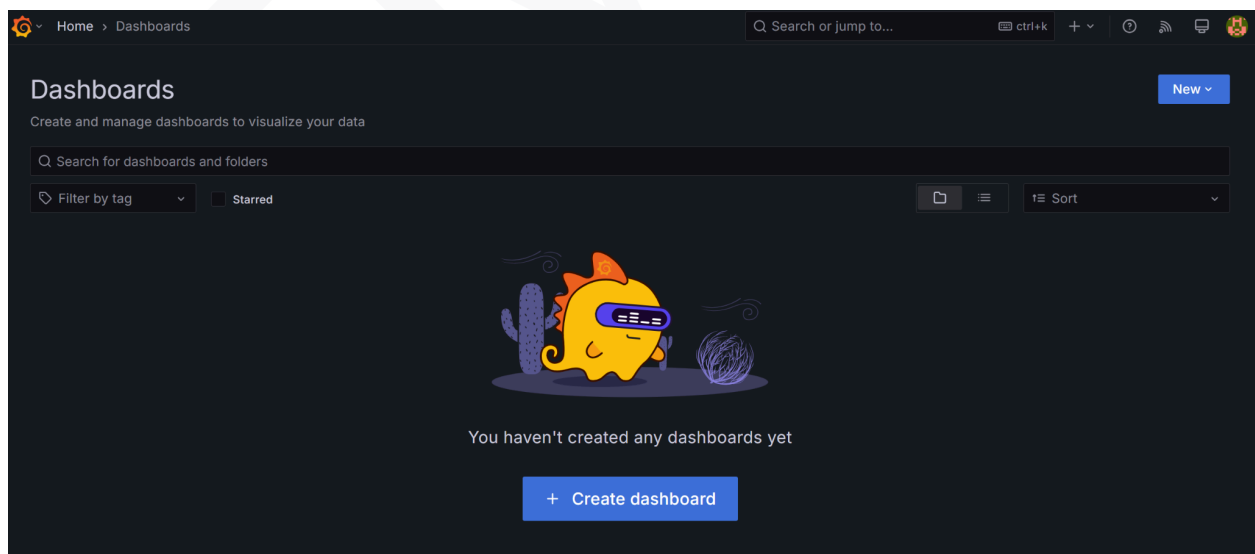
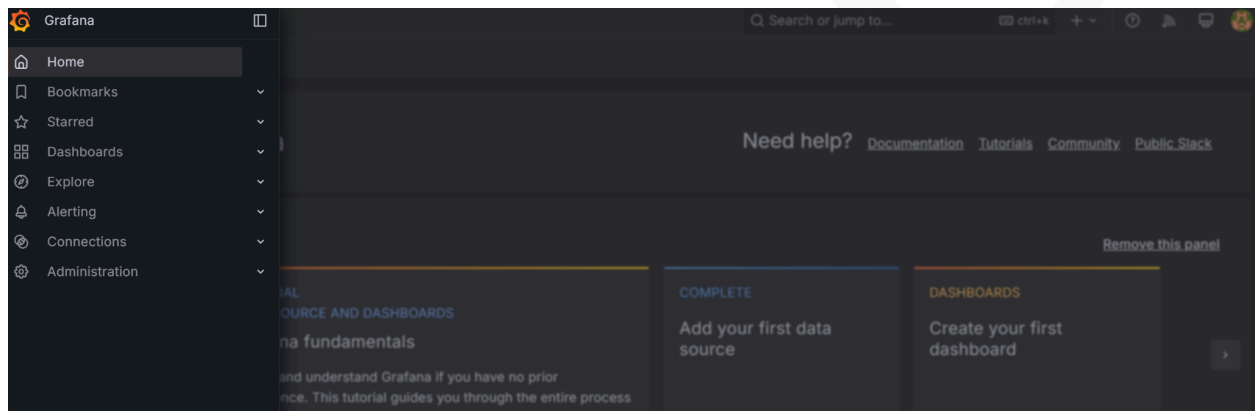
1. FastAPI: <http://localhost:8000/docs#/>
2. Grafana: <http://localhost:3000/login>
 - a. The default credentials are user: admin and password: admin
3. Prometheus: <http://localhost:9090/query>

default

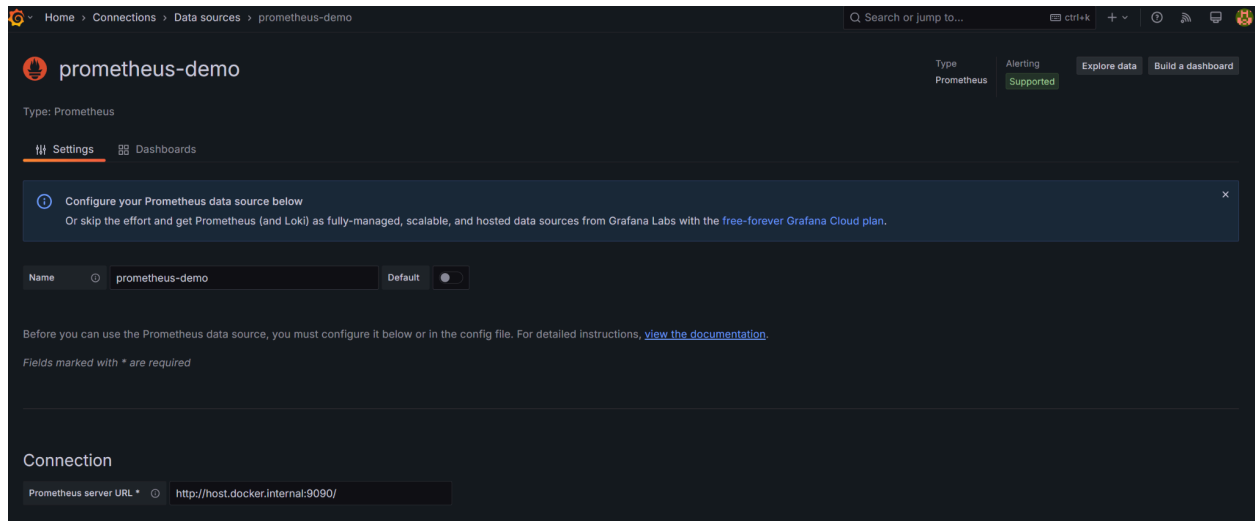
GET	/metrics	Metrics	▼
GET	/	Read Root	▼
POST	/predict	Read Root	▼

FastAPI will show metrics which is from the Prometheus instrumentator. Try to make random requests to FastAPI endpoints.

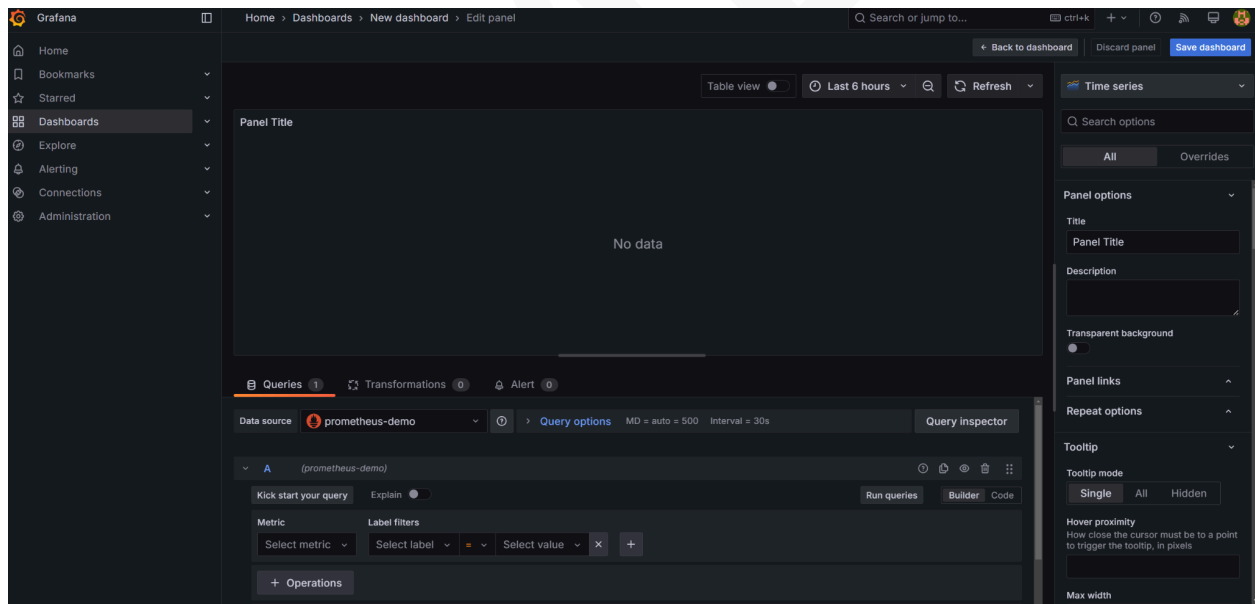
Now go to your Grafana and open the Dashboard



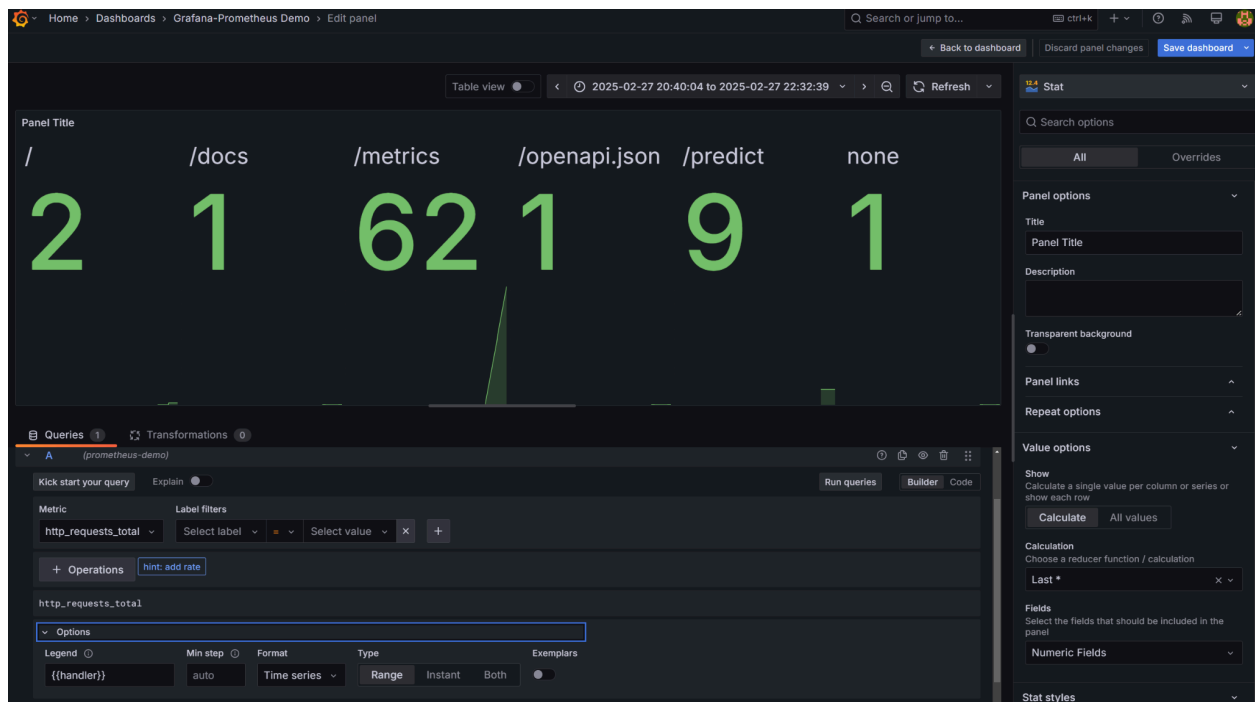
Create Dashboard -> Add Visualization -> Configure New Data Source
-> Paste `http://host.docker.internal:9090/` -> Scroll down and click Save
& Test



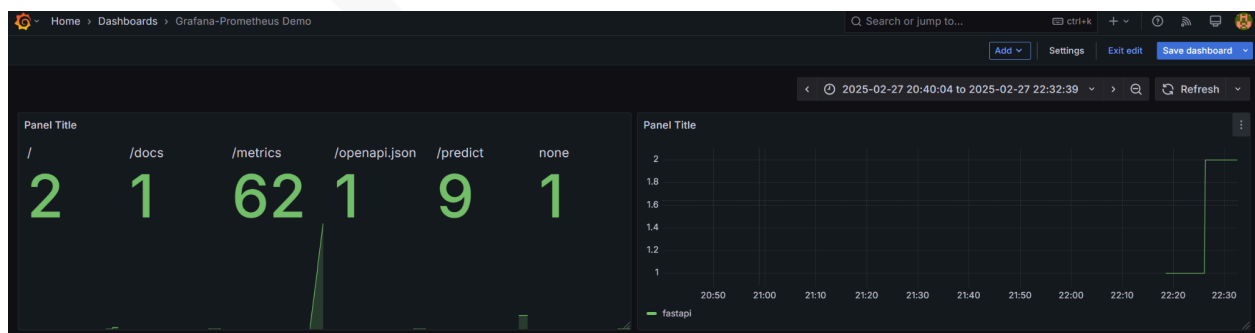
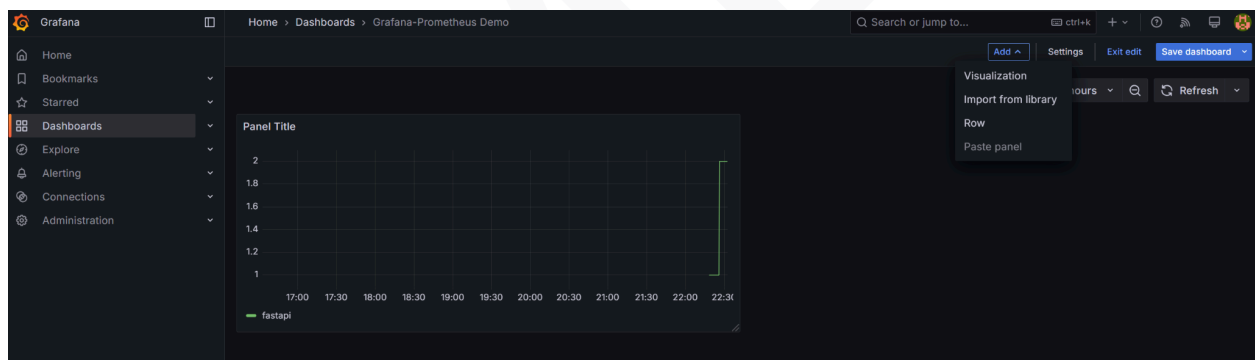
Then again go back to Dashboard and select the Data Source you defined



Below there are options to set up queries and configure data as a graph



Once queries are added, save the Dashboard. You can go back add more visualization panels



Conclusion

In LLMOps, **Grafana and Prometheus** form the backbone of model observability. With Prometheus collecting granular metrics and Grafana providing intuitive visualization, AI teams can ensure **scalable, efficient, and reliable** LLM deployments.

By integrating these tools into your AI workflow, you gain **proactive monitoring, real-time insights, and better resource optimization**, ensuring that your Generative AI applications run smoothly and cost-effectively.

TNMG